AR Mining On Streams

Data streams

Stream of data arrive in rapid succession, no Re-scan is, storage space is insufficient to accommodate all data points.

Without storing all the data one wish to estimate

- Set of frequent items
- Number of distinct items
- Frequent itemsets
- *etc*

[Constraints are on memory and processing power]

Frequent pattern mining over data streams

- Applications involves retail market data analysis, network monitoring, web usage mining, and stock market prediction.
- Using sliding window
- Efficiently remove the obsolete, old stream data
- Compact Pattern Stream tree (CPS-tree) [1]
- Highly compact frequency-descending tree structure at runtime
- Efficient in terms of memory and time complexity
- Pane and window
- Insertion and restructuring

[1] Tanbeer, Syed Khairuzzaman and Ahmed, Chowdhury Farhan and Jeong, Byeong-Soo and Lee, Young-Koo, "Sliding window-based frequent pattern mining over data streams", in Information sciences, 179(22) pages 3843-3865, Elsevier, 2009

CPS-tree construction

Algorithm 1 *(Construction of a CPS-tree for a data stream)*
Input: *Stream_data, Pane_size, Window_size, Initial_Sort_Order*
Output: T_{sort}: *a CPS-tree for the current window*
Method:

```
    Begin
1:      w ← ∅;
2:      T ← a prefix-tree with null initialization;
3:      Current_Sort_Order ← Initial_Sort_Order;
    //For the first Window
4:          While (w ≠ Window_size) do
5:              Call Insert_Pane(T);                                  // Insertion Phase
6:              Current_Sort_Order ← Frequency-descending sort order; // Restructuring Phase
7:              Restructure T;
8:              w = w+1;
9:          End While
    //At each slide of Window
10:         Repeat
11:             Delete the oldest pane information from T;            // Extracting the old pane
12:             Call Insert_Pane(T);                                  // Insertion Phase
13:             Current_Sort_Order ← Frequency-descending sort order; // Restructuring Phase
14:             Restructure T;
15:         End
    End

Insert_Pane(Tr)
    Begin
1:      p ← ∅;
2:      While (p ≠ Pane_size) do
3:          Scan transaction from the current location in Stream_data;
4:          Insert the scanned transaction into Tr according to Current_Sort_Order;
5:          p = p+1;
6:      End While
    End
```

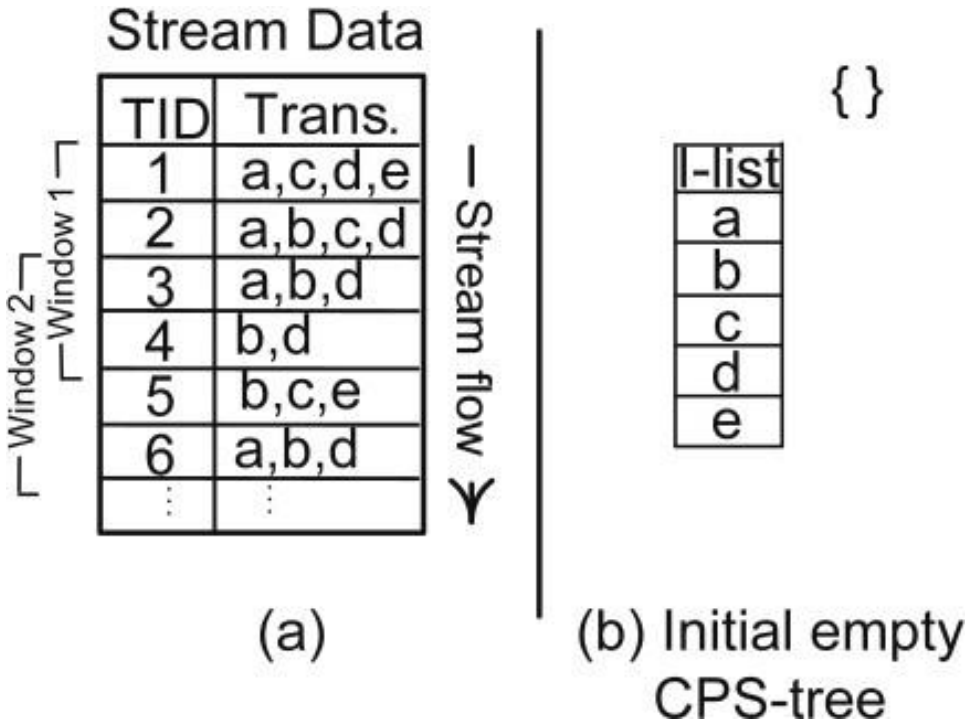
Fig. 3. The CPS-tree construction algorithm.

Algorithm 2 *(Extracting old pan information)*
Input: T_{sort}: *a CPS-tree for the previous window*
Output: *T: a prefix-tree ready to capture new pane information*
Method:

```
    Begin
1:      For each item i from the bottom of I_sort
2:          For each node N_i for i in T_sort
3:              If (N_i is a tail-node)
4:                  If (N_i.v_1 > 0)
5:                      N_i.s = N_i.s - N_i.v_1;
6:                      If (N_i.s = 0)
7:                          Delete N_i;
8:                      Else
9:                          Shift each v_j of N_i to left for one slot and insert a zero at the last entry;
10:                     End If
11:                     Update I_sort for N_i;
12:                     For each node N_k between N_i and the root
13:                         N_k.s = N_k.s - N_i.v_1;
14:                         If (N_k.s = 0)
15:                             Delete N_k;
16:                         End If
17:                         Update I_sort for N_k;
18:                     End For
19:                 End If
20:                 Shift each v_j of N_i to left for one slot and insert a zero at the last entry;
21:             End If
22:         End For
23:     End For
    End
```

Fig. 4. The pane extraction algorithm for the CPS-tree.

CPS-tree construction



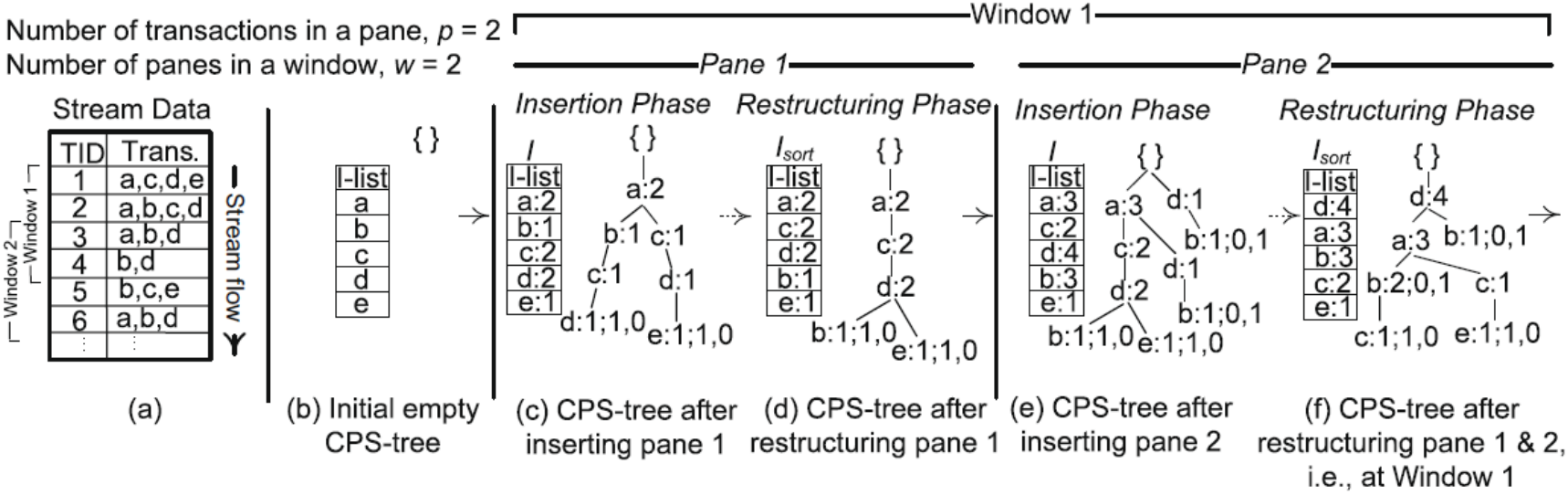


Fig. 2. Construction of the CPS-tree.

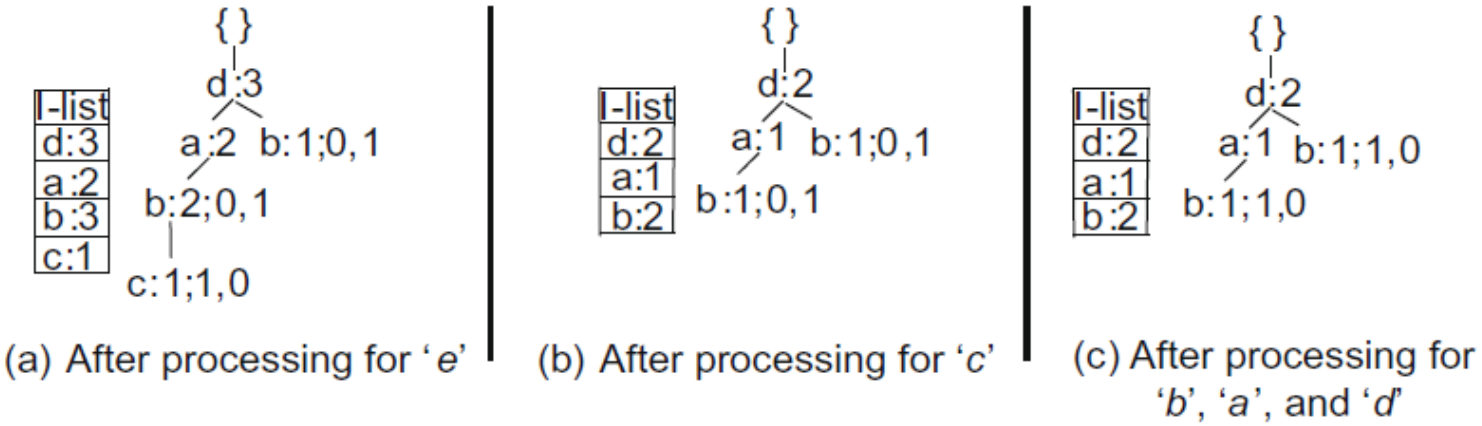
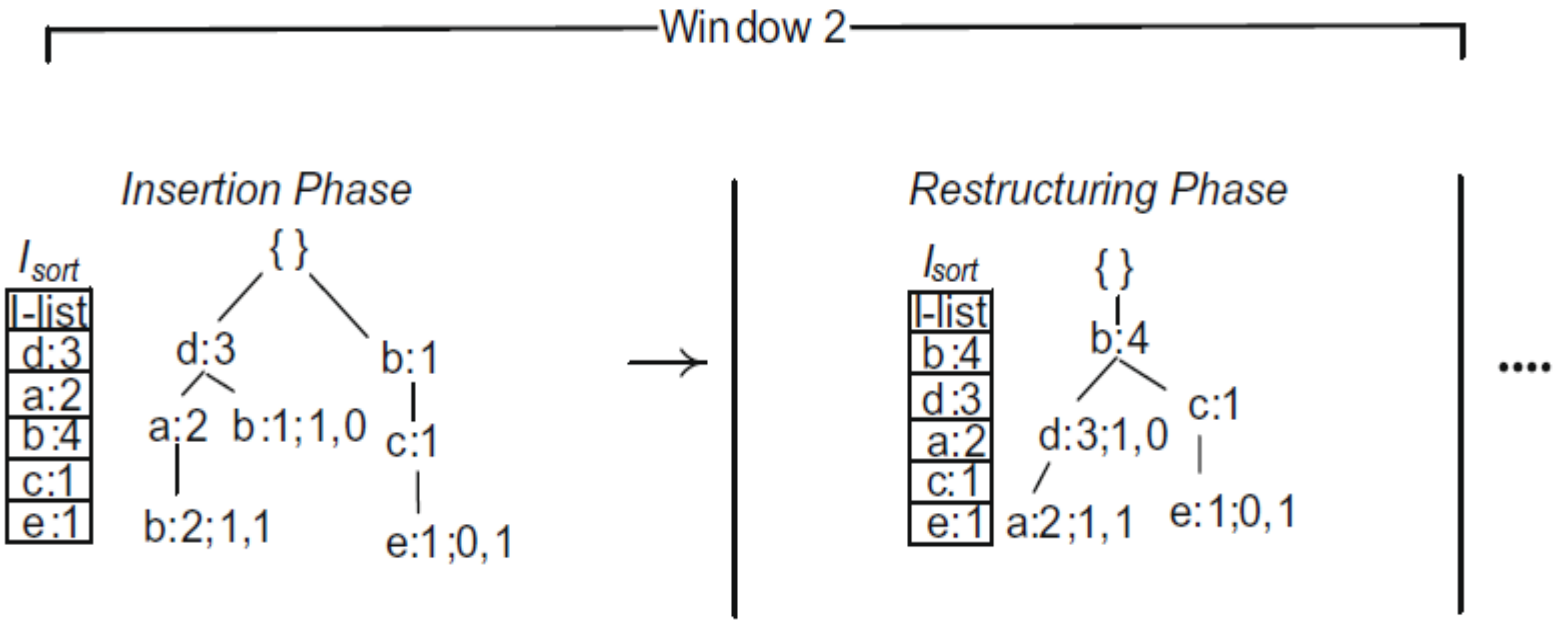


Fig. 5. Extracting old pane from CPS-tree of Fig. 2f.



(a) CPS-tree after inserting new pane (i.e., *tid*s 5 and 6) at Window 2

(b) CPS-tree after restructuring at Window 2

Fig. 6. The CPS-tree in Window 2 for the stream data of Fig. 2a.